

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Долдин Александр Дмитриевич
(подпись) (фамилия, имя, отчество)

Группы ИПс-21

Руководитель практической подготовки от профильной организации

Преподаватель
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ	5
1.1. Сбор информации о существующем состоянии продукта	5
1.2. Разработка технического задания	10
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ	15
2.1. Процесс разработки программного обеспечения	15
2.2 Разработка программного модуля	17
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ.....	36
3.1. Процесс отладки	36
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	40
4.1. Процесс тестирования.....	40
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	44
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.....	44
ЗАКЛЮЧЕНИЕ.....	49
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	51
Требование к написанию программного модуля	52

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности
12. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ

1.1. Сбор информации о существующем состоянии продукта

Постановка задач

В рамках летней практики была поставлена задача разработать приложение для управления задачами (TaskManagerApp), предназначенное для автоматизации процессов планирования, отслеживания и контроля выполнения задач.

Основные задачи, которые необходимо решить:

1. Создать удобный инструмент для добавления и ведения списка задач с возможностью указания названия, описания, приоритета, срока выполнения и статуса.
2. Реализовать систему фильтрации и поиска задач для быстрого доступа к необходимой информации.
3. Обеспечить возможность редактирования и удаления существующих задач.
4. Организовать сохранение данных в файл и их загрузку для обеспечения сохранности информации между сеансами работы.
5. Реализовать графический интерфейс для взаимодействия пользователя с системой.
6. Разработать модульные тесты для проверки корректности работы основной логики приложения.

Описание бизнес-процесса (состояние «AS IS»)

На текущий момент процесс управления задачами осуществляется неавтоматизированным способом, а именно:

1. Постановка задачи: Задача формулируется устно или в текстовом сообщении (электронная почта, мессенджер).
2. Фиксация: Информация о задаче записывается в блокнот, файл Microsoft Excel, текстовый документ или просто хранится в памяти исполнителя.
3. Отслеживание статуса: Для контроля выполнения задачи необходимо уточнять информацию у исполнителя, просматривать переписку или проверять рукописные записи.
4. Поиск информации: При необходимости найти задачу требуется вручную просматривать все записи, что занимает значительное время, особенно при большом количестве задач.
5. Обновление данных: При изменении приоритета, срока или статуса задачи информацию необходимо заново переписывать или редактировать вручную.
6. Сохранение данных: Отсутствует единое централизованное хранилище задач — информация распределена по разным источникам.

Описание процесса модернизации (работа системы после модернизации – состояние «ТО ВЕ»)

После внедрения разрабатываемого приложения «TaskManagerApp» процесс управления задачами становится автоматизированным.

1. Постановка задачи: Задача вносится через графический интерфейс с заполнением всех необходимых полей (название, описание, приоритет, срок, статус).
2. Фиксация: Информация структурированно сохраняется в оперативной памяти и дублируется в JSON-файл для сохранности.

3. Отслеживание статуса: Статус задачи отображается в общем списке, доступна фильтрация по статусам («Новая», «В процессе», «Завершена»).

4. Поиск информации: Реализован поиск по названию и описанию задачи, позволяющий мгновенно находить необходимую информацию.

5. Обновление данных: Через интерфейс можно быстро отредактировать любую задачу — изменить название, описание, приоритет, срок, статус или отметить как важную.

6. Сохранение данных: Все задачи автоматически сохраняются в файл формата JSON и загружаются при следующем запуске приложения.

7. Статистика: В приложении реализован просмотр статистики — количество завершённых и просроченных задач.

Как будут решаться задачи, которые необходимо автоматизировать

Таблица 1 — Задачи для автоматизации

№ Задача для автоматизации	Способ решения
1 Добавление новых задач	Реализована форма ввода с полями: название, описание, приоритет, срок выполнения, статус, отметка о важности.
2 Хранение и систематизация задач	Все задачи хранятся в коллекции в оперативной памяти и сериализуются в JSON-файл.
3 Поиск задач	Реализован через LINQ-запрос по частичному совпадению в названии или описании.
4 Фильтрация задач	Реализована через LINQ-запрос с выбором статуса из выпадающего списка.
5 Редактирование задач	Реализовано через открытие формы с

	подгруженными данными задачи и их перезаписью.
6 Удаление задач	Реализовано через подтверждение удаления и удаление элемента из коллекции.
7 Контроль просроченных задач	Реализован метод в модели задачи для сравнения срока с текущей датой; подсчёт в статистике.
8 Сохранение данных между сеансами	Реализована запись в JSON-файл при нажатии кнопки "Сохранить" и загрузка при нажатии "Загрузить".
9 Визуализация важных задач	Задача отмечается звёздочкой ★ в списке при выставлении флага "Важная".
10 Статистика выполнения	Реализован подсчёт завершённых и просроченных задач через методы LINQ.

Таблица 2 — Перечень действий процессов «AS IS» и «TO BE»

Критерий	Состояние «AS IS»	Состояние «TO BE»	Что усовершенствовано
Способ добавления задач	Вручную в блокноте/Excel	Через специальную форму интерфейсе	Автоматизирован ввод, структурированные поля
Хранение данных	Разрозненно, нет единой системы	В единой коллекции с сохранением в JSON	Централизованное хранение с возможностью восстановления
Поиск задач	Вручную, просмотр	Автоматически по ключевым словам	Экономия времени, точность

	записей		поиска
Фильтрация	Отсутствует	По статусу через выпадающий список	Быстрый доступ к задачам определенного статуса
Редактирование	Переписывание вручную	Через форму редактирования	Простота внесения изменений
Отслеживание статуса	Уточнение исполнителя	уВизуально в списке задач	Актуальная информация всегда доступна
Сохранность данных	Может быть потеряна	Автосохранение в JSON	Данные не теряются после закрытия приложения
Важные задачи	Не выделяются	Отмечаются ★	Визуальное выделение приоритета
Статистика	Отсутствие	Автоматические подсчет завершенных/прос роченных	Оценка эффективности работы
Трудовые затраты	Высокие (много ручных операций)	Низкие (все автоматизировано)	Повышение производительности
Вероятность ошибок	Высокая	Низкая	Исключение человеческого фактора

1.2. Разработка технического задания

Основные требования к техническому заданию

Техническое задание (ТЗ) является основополагающим документом, определяющим цели, требования и порядок разработки программного продукта. К разработке ТЗ предъявляются следующие требования:

1. Полнота — ТЗ должно содержать все необходимые сведения для разработки, исключая двоякое толкование.
2. Однозначность — все формулировки должны быть чёткими и не допускать разночтений.
3. Проверяемость — каждое требование должно допускать возможность проверки его выполнения.
4. Непротиворечивость — требования не должны противоречить друг другу.
5. Актуальность — ТЗ должно отражать реальные потребности пользователя.
6. Детализация — достаточный уровень проработки для реализации.

Стандарты, используемые для написания технического задания

Для написания технического задания использовались следующие стандарты:

Таблица 3 — Используемые стандарты

Документ	Назначение
ГОСТ 19.101-77	Виды программных документов (ТЗ, технический проект, пояснительная записка и др.)
ГОСТ 19-201-78	Требования к оформлению ТЗ на разработку программного обеспечения

ГОСТ 19.401-79	Требования к пояснительной записке
ГОСТ 34.602-89	Техническое задание на создание автоматизированной системы
ГОСТ 34.601-90	Стадии создания автоматизированных систем
ГОСТ 34.603-92	Виды испытаний автоматизированных систем

Содержание технического задания

Техническое задание включает следующие разделы:

1. Введение — цели и задачи для разработки;
2. Основание для разработки — разработка осуществляется на основании задания на практику;
3. Назначение разработки — приложение предназначено для автоматизации процессов планирования, отслеживания и контроля выполнения задач с возможностью их фильтрации, поиска и редактирования;

Таблица 4 — Требования к программе

№	Функция	Описание
1	Добавление задачи	Пользователь вводит название, описание, приоритет, срок и статус задачи
2	Просмотр списка	Отображение всех задач с их параметрами
3	Фильтрация	Выбор задач по статусу через выпадающий список
4	Поиск	Поиск по названию или описанию задачи
5	Редактирование	Изменение параметров существующей задачи
6	Удаление	Удаление задачи с подтверждением
7	Сохранение/Загрузка	Сериализация в JSON-формат и десериализация из файла
8	Отметка	Возможность пометить задачу как важную

9 Статистика Подсчет завершенных и просроченных задач

4. Требования к программной документации — отчет по практике, дневник практики и аттестационный лист, исходные коды программы;

5. Техничко-экономические показатели — приложение разрабатывается для учебных целей, минимальные требования: Windows 10/11, .NET 8.0;

Таблица 5 — Стадии и этапы разработки:

№ Этап	Описание	Сроки
1 Анализ требований	Изучение задания, определение требований	1-2 день
2 Проектирование	Разработка архитектуры	3 день
3 Разработка библиотеки	Создание классов, реализация логики	4-6 дни
4 Разработка интерфейса	Создание WPF-приложения	7-9 дни
5 Написание тестов	Разработка unit-тестов	10 день
6 Отладка и тестирование	Проверка работоспособности	11-12 дни
7 Документирование	Оформление отчета	13-14 дни

6. Порядок контроля и приёмки — проверка работоспособности приложения, проверка запуска модульных тестов, наличие всех элементов документации.

Структура технического задания

На основе ГОСТ 19.201-78 и ГОСТ 34.602-89 структура ТЗ включает:

1. Общие сведения

- Наименование разработки: Приложение для управления задачами «TaskManagerApp»

- Заказчик: Пермский институт (филиал) Им. Г.В.Плеханова
- Дата утверждения: 2025 г.
- Разработчик: Студент ИПс-21 Долдин Александр Дмитриевич

2. Назначение и цели разработки

- Цель: Разработать приложение для управления задачами с графическим интерфейсом, удовлетворяющее требованиям задания.
- Назначение: Автоматизация процессов планирования и контроля выполнения задач.

3. Характеристика объекта автоматизации

- Пользователь: Студент / руководитель практики.
- Текущий процесс: Ручное управление задачами (блокнот, Excel, устные поручения).
- Проблемы: Отсутствие структурированности, сложность поиска, потеря данных.

4. Требования к системе

- Функциональные: добавление, просмотр, фильтрация, поиск, редактирование, удаление, сохранение/загрузка, статистика.
- Технические: разработать в среде Visual Studio, язык C#, WPF, .NET 8.0, JSON для хранения.
- Эргономические: русскоязычный интерфейс, интуитивная навигация.

5. Состав и содержание работ

- Разработка архитектуры (DLL + WPF)
- Реализация бизнес-логики
- Разработка UI
- Написание тестов
- Документирование

6. Порядок контроля и приёмки

- Демонстрация работы приложения
- Запуск всех тестов

- Проверка отчёта

Порядок документирования требований

Процесс документирования требований включает следующие этапы:

1. Сбор требований

- Анализ задания на практику.
- Выявление потребностей.
- Определение необходимого функционала.

2. Анализ и структурирование

- Классификация требований (функциональные, нефункциональные, технические).

- Приоритизация требований (обязательные и дополнительные).
- Проверка на противоречивость.

3. Формализация

- Запись требований в структурированной форме.
- Использование таблиц для наглядности.
- Описание сценариев использования.

4. Валидация (проверка правильности)

- Проверка: каждое требование выполнимо?
- Проверка: требования полные и непротиворечивые?

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Выбор модели разработки

Для разработки приложения «TaskManagerApp» была выбрана каскадная (водопадная) модель по следующим причинам:

1. Чётко определённые требования — все функциональные требования известны заранее (из задания на практику) и не предполагают существенных изменений в процессе разработки.
2. Небольшой объём проекта — приложение не является масштабным, не требует сложного управления рисками или длительных итераций.
3. Образовательный характер — практика направлена на закрепление знаний. Каждый этап логически завершён и может быть документирован.
4. Простота планирования — все этапы (анализ, проектирование, реализация, тестирование, документирование) выполняются последовательно, это довольно удобно.
5. Документирование на каждом этапе — позволяет подготовить полноценную сопроводительную записку, что является обязательным требованием практики.

Таблица 6 — Инструменты, использованные для создания продукта

Инструмент	Назначение
Microsoft Visual Studio 2026	Интегрированная среда разработки (IDE)
.NET 8.0	Платформа для разработки и выполнения приложения

C#	Язык программирования
WPF	Фреймворк для создания графического интерфейса
xUnit	Фреймворк для написания модульных тестов
Git	Система контроля версий
GitHub	Хостинг репозитория для хранения кода
Draw.io	Построение диаграммы классов
Microsoft Word	Оформление отчета

Среда программирования

В качестве среды разработки используется Microsoft Visual Studio 2026.

Таблица 7 — Ключевые характеристики

Параметр	Описание
Название	Visual Studio 2026 Insiders
Версия	11904.113
Тип	Бесплатная интегрированная среда разработки
Поддерживаемые языки	C#, C++. Python, JavaScript
Поддерживаемые платформы	.NET, WPF, WinForms

Почему выбрана именно эта среда:

1. Бесплатна и широко распространена;
2. Поддерживает все необходимые технологии (.NET 8.0, WPF, xUnit);
3. Имеет мощный встроенный отладчик;
4. Интегрирована с Git и GitHub.

Диаграмма классов


```

public string Title { get; set; }

public string Description { get; set; }

public TaskPriority Priority { get; set; }

public DateTime DueDate { get; set; }

public MyTaskStatus Status { get; set; }

public bool IsImportant { get; set; }

public TaskItem() { }

public TaskItem(int id, string title, string description,
                TaskPriority priority, DateTime dueDate,
                MyTaskStatus status, bool isImportant = false)
{
    Id = id;
    Title = title;
    Description = description;
    Priority = priority;
    DueDate = dueDate;
    Status = status;
    IsImportant = isImportant;
}

public bool IsOverdue()
{

```

```

        return Status != MyTaskStatus.Завершена && DueDate <
DateTime.Today;
    }

    public override string ToString()
    {
        string star = IsImportant ? " ★" : "";

        string statusText = Status.ToString();
        if (statusText == "ВПроцессе")
            statusText = "В процессе";

        return $"[{Id}] {Title} ({Priority}, {statusText}){star} - до
{DueDate:yyyy-MM-dd}";
    }
}
}

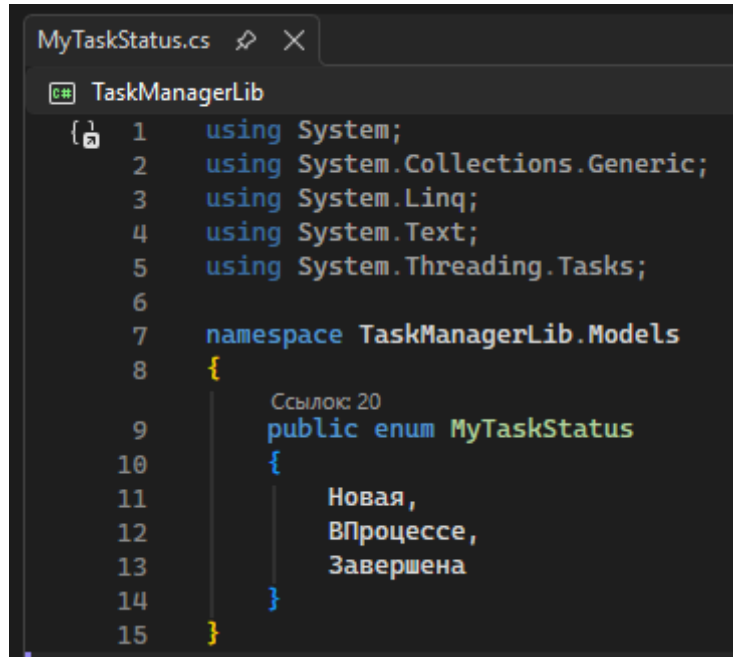
```

```
TaskItem.cs  ✎  ✕
TaskManagerLib
TaskManagerLib.Models.

1  using System;
2
3  namespace TaskManagerLib.Models
4  {
5      Ссылка: 27
6      public class TaskItem
7      {
8          Ссылка: 9
9          public int Id { get; set; }
10
11          Ссылка: 11
12          public string Title { get; set; }
13
14          Ссылка: 7
15          public string Description { get; set; }
16
17          Ссылка: 6
18          public TaskPriority Priority { get; set; }
19
20          Ссылка: 7
21          public DateTime DueDate { get; set; }
22
23          Ссылка: 10
24          public MyTaskStatus Status { get; set; }
25
26          Ссылка: 6
27          public bool IsImportant { get; set; }
28
29          Ссылка: 1
30          public TaskItem() { }
31
32          Ссылка: 9
33          public TaskItem(int id, string title, string description,
34                          TaskPriority priority, DateTime dueDate,
35                          MyTaskStatus status, bool isImportant = false)
36          {
37              Id = id;
38              Title = title;
39              Description = description;
40              Priority = priority;
41              DueDate = dueDate;
42              Status = status;
43              IsImportant = isImportant;
44          }
45
46          Ссылка: 1
47          public bool IsOverdue()
48          {
49              return Status != MyTaskStatus.Завершена && DueDate < DateTime.Today;
50          }
51
52          Ссылка: 0
53          public override string ToString()
54          {
55              string star = IsImportant ? " ★ " : "";
56
57              string statusText = Status.ToString();
58              if (statusText == "В процессе")
59                  statusText = "В процессе";
60
61              return $"[{Id}] {Title} ({Priority}, {statusText}){star} - до {DueDate:yyyy-MM-dd}";
62          }
63      }
64  }
```

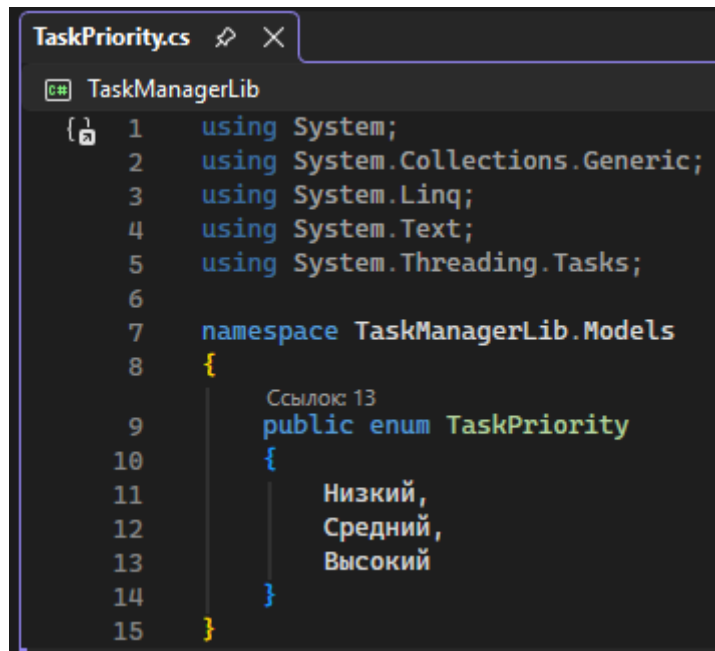
Рисунок 2 - TaskItem.cs

Реализация перечислений

The image shows a code editor window titled 'MyTaskStatus.cs'. The code is in C# and defines a namespace 'TaskManagerLib.Models'. Inside this namespace, there is a public enum 'MyTaskStatus' with three values: 'Новая', 'ВПроцессе', and 'Завершена'. The code includes several 'using' statements at the top: 'using System;', 'using System.Collections.Generic;', 'using System.Linq;', 'using System.Text;', and 'using System.Threading.Tasks;'. The enum is defined on line 9, and its values are listed on lines 11, 12, and 13. The code is enclosed in a namespace block from line 7 to line 15.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace TaskManagerLib.Models
8 {
9     Ссылка: 20
    public enum MyTaskStatus
    {
11         Новая,
12         ВПроцессе,
13         Завершена
14     }
15 }
```

Рисунок 3 - MyTaskStatus.cs

The image shows a code editor window titled 'TaskPriority.cs'. The code is in C# and defines a namespace 'TaskManagerLib.Models'. Inside this namespace, there is a public enum 'TaskPriority' with three values: 'Низкий', 'Средний', and 'Высокий'. The code includes several 'using' statements at the top: 'using System;', 'using System.Collections.Generic;', 'using System.Linq;', 'using System.Text;', and 'using System.Threading.Tasks;'. The enum is defined on line 9, and its values are listed on lines 11, 12, and 13. The code is enclosed in a namespace block from line 7 to line 15.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace TaskManagerLib.Models
8 {
9     Ссылка: 13
    public enum TaskPriority
    {
11         Низкий,
12         Средний,
13         Высокий
14     }
15 }
```

Рисунок 4 - TaskPriority.cs

Реализация класса TaskManager

Класс TaskManager реализует бизнес-логику приложения.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Text.Json;
using TaskManagerLib.Models;

namespace TaskManagerLib
{
    public class TaskManager
    {
        private List<TaskItem> _tasks = new List<TaskItem>();

        private int _nextId = 1;

        public List<TaskItem> GetTasksByStatus(MyTaskStatus status)
        {
            return _tasks.Where(t => t.Status == status).ToList();
        }

        public List<TaskItem> SearchTasks(string keyword)
        {
            if (string.IsNullOrEmpty(keyword))
                return _tasks.ToList();

            return _tasks.Where(t => t.Title.Contains(keyword,
StringComparison.OrdinalIgnoreCase) ||
                t.Description.Contains(keyword,
StringComparison.OrdinalIgnoreCase))
                .ToList();
        }
    }
}

```

```

    }

    public (int completed, int overdue) GetStatistics()
    {
        int completed = _tasks.Count(t => t.Status ==
MyTaskStatus.Завершена);
        int overdue = _tasks.Count(t => t.IsOverdue());
        return (completed, overdue);
    }

    public void AddTask(TaskItem task)
    {
        task.Id = _nextId++;
        _tasks.Add(task);
    }

    public bool UpdateTask(int id, string newTitle, string newDescription,
        TaskPriority newPriority, DateTime newDueDate,
        MyTaskStatus newStatus, bool newIsImportant)
    {
        var task = _tasks.FirstOrDefault(t => t.Id == id);
        if (task == null) return false;

        task.Title = newTitle;
        task.Description = newDescription;
        task.Priority = newPriority;
        task.DueDate = newDueDate;
        task.Status = newStatus;
        task.IsImportant = newIsImportant;
        return true;
    }

```

```
}
```

```
public bool DeleteTask(int id)
{
    var task = _tasks.FirstOrDefault(t => t.Id == id);
    if (task == null) return false;

    return _tasks.Remove(task);
}
```

```
public List<TaskItem> GetAllTasks()
{
    return _tasks.ToList();
}
```

```
public void SaveToFile(string filePath)
{
    string json = JsonSerializer.Serialize(new
    {
        NextId = _nextId,
        Tasks = _tasks
    }, new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(filePath, json);
}
```

```
public void LoadFromFile(string filePath)
{
    if (!File.Exists(filePath)) return;

    string json = File.ReadAllText(filePath);
}
```



```

var data = JsonSerializer.Deserialize<SaveData>(json);
if (data != null)
{
    _nextId = data.NextId;
    _tasks = data.Tasks ?? new List<TaskItem>();
}
}

private class SaveData
{
    public int NextId { get; set; }
    public List<TaskItem> Tasks { get; set; }
}
}
}

```

```

TaskManagerLib
TaskManagerLib.TaskManager

10  Ссылка 7
11  public class TaskManager
12  {
13      private List<TaskItem> _tasks = new List<TaskItem>();
14
15      private int _nextId = 1;
16
17      Ссылка 1
18      public List<TaskItem> GetTasksByStatus(MyTaskStatus status)
19      {
20          return _tasks.Where(t => t.Status == status).ToList();
21      }
22
23      Ссылка 1
24      public List<TaskItem> SearchTasks(string keyword)
25      {
26          if (string.IsNullOrEmpty(keyword))
27              return _tasks.ToList();
28
29          return _tasks.Where(t => t.Title.Contains(keyword, StringComparison.OrdinalIgnoreCase) ||
30              t.Description.Contains(keyword, StringComparison.OrdinalIgnoreCase))
31              .ToList();
32      }
33
34      Ссылка 2
35      public (int completed, int overdue) GetStatistics()
36      {
37          int completed = _tasks.Count(t => t.Status == MyTaskStatus.Завершена);
38          int overdue = _tasks.Count(t => t.IsOverdue());
39          return (completed, overdue);
40      }
41
42      Ссылка 10
43      public void AddTask(TaskItem task)
44      {
45          task.Id = _nextId++;
46          _tasks.Add(task);
47      }
48
49      Ссылка 1
50      public bool UpdateTask(int id, string newTitle, string newDescription,
51          TaskPriority newPriority, DateTime newDueDate,
52          MyTaskStatus newStatus, bool newIsImportant)
53      {
54          var task = _tasks.FirstOrDefault(t => t.Id == id);
55          if (task == null) return false;
56
57          task.Title = newTitle;
58          task.Description = newDescription;
59          task.Priority = newPriority;
60          task.DueDate = newDueDate;
61          task.Status = newStatus;

```

Рисунок 5 - отрывок TaskManager.cs

Этапы разработки ПО

Выбор шаблона архитектуры

Разработанное приложение построено на архитектурном шаблоне Model-View-Controller (MVC) в упрощенной интерпретации.

Таблица 8 - Компоненты

Компонент Реализация

Назначение

Model	TaskItem, TaskManager	Бизнес-логика, данные, правила
View	MainWindow, TaskEditWindow	Отображение данных и взаимодействие с пользователем
Controller	Код в MainWindow.xaml.cs	Обработка событий, связь модели и представления

Составление схемы информационной базы

В приложении не используется реляционная база данных; вместо этого применяется структурированное хранилище в виде коллекций и JSON-файла.

Схема данных:

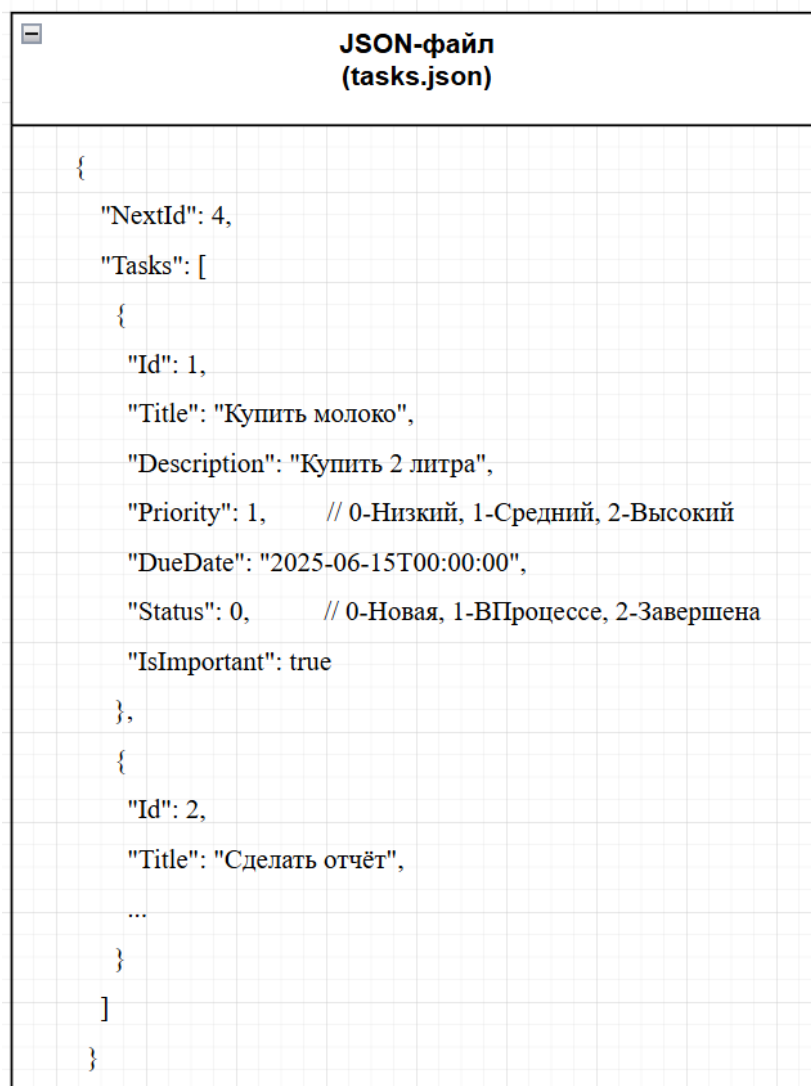


Рисунок 6 - Схема данных

Документирование системы

Документирование выполнялось на всех этапах разработки

Таблица 9 — Документирование

Этап	Документ	Содержание
Анализ	Техническое задание	Требования, функции, ограничения
Проектирование	Диаграмма классов	Структура системы, связи между классами
Реализация	Исходный код	Код программы
Тестирование	Модульные тесты	xUnit-тесты
Итог	Отчет	Описание процесса разработки

Тестирование

Тестирование проводилось с помощью модульного тестирования (Unit-тестов):

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using TaskManagerLib;  
using TaskManagerLib.Models;  
using Xunit;
```

```
namespace TaskManagerTests  
{
```

```

public class TaskManagerTests
{
    [Fact]
    public void AddTask_ShouldIncreaseCount()
    {
        var manager = new TaskManager();
        var task = new TaskItem(0, "Тест", "Описание",
TaskPriority.Средний, DateTime.Today, MyTaskStatus.Новая);

        manager.AddTask(task);
        var all = manager.GetAllTasks();

        Assert.Single(all);
        Assert.Equal(1, all[0].Id);
    }

    [Fact]
    public void FilterByStatus_ShouldReturnCorrect()
    {
        var manager = new TaskManager();
        manager.AddTask(new TaskItem(0, "А", "", TaskPriority.Низкий,
DateTime.Today, MyTaskStatus.Новая));
        manager.AddTask(new TaskItem(0, "В", "", TaskPriority.Низкий,
DateTime.Today, MyTaskStatus.Завершена));

        var filtered = manager.GetTasksByStatus(MyTaskStatus.Новая);

        Assert.Single(filtered);
        Assert.Equal("А", filtered[0].Title);
    }
}

```

```

[Fact]
public void SearchTasks_ByTitle_ShouldFind()
{
    var manager = new TaskManager();
    manager.AddTask(new TaskItem(0, "Купить молоко", "",
TaskPriority.Средний, DateTime.Today, MyTaskStatus.Новая));
    manager.AddTask(new TaskItem(0, "Позвонить", "",
TaskPriority.Средний, DateTime.Today, MyTaskStatus.Новая));

    var found = manager.SearchTasks("молоко");

    Assert.Single(found);
    Assert.Contains("молоко", found[0].Title);
}

```

```

[Fact]
public void DeleteTask_ShouldRemove()
{
    var manager = new TaskManager();
    manager.AddTask(new TaskItem(0, "Задача", "",
TaskPriority.Низкий, DateTime.Today, MyTaskStatus.Новая));
    int id = manager.GetAllTasks()[0].Id;

    bool deleted = manager.DeleteTask(id);

    Assert.True(deleted);
    Assert.Empty(manager.GetAllTasks());
}

```

[Fact]

```
public void Statistics_CompletedAndOverdue()
{
    var manager = new TaskManager();
    manager.AddTask(new TaskItem(0, "1", "", TaskPriority.Низкий,
DateTime.Today.AddDays(-1), MyTaskStatus.Новая));
    manager.AddTask(new TaskItem(0, "2", "", TaskPriority.Низкий,
DateTime.Today, MyTaskStatus.Завершена));
    manager.AddTask(new TaskItem(0, "3", "", TaskPriority.Низкий,
DateTime.Today.AddDays(1), MyTaskStatus.Новая));

    var stats = manager.GetStatistics();

    Assert.Equal(1, stats.completed);
    Assert.Equal(1, stats.overdue);
}
}
```

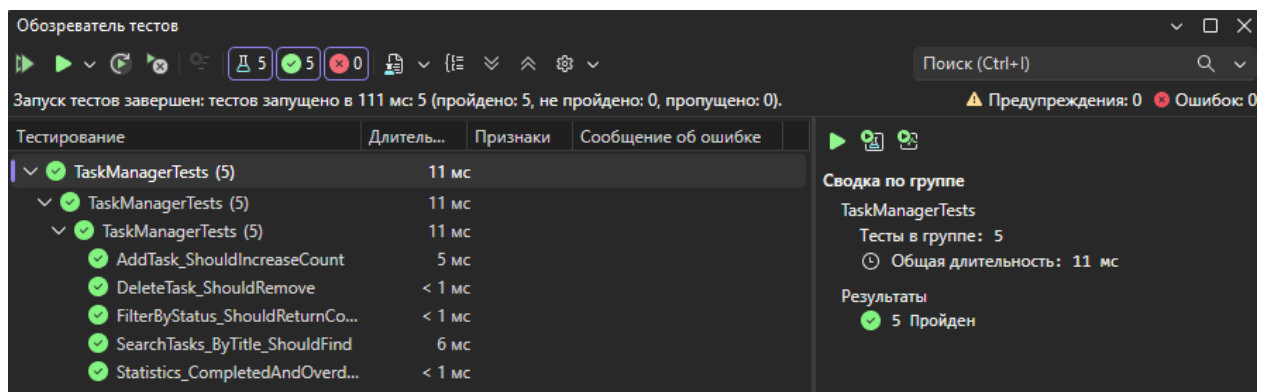


Рисунок 7 - Запуск тестов

}

Сопровождение системы

После завершения разработки выполняется сопровождение в виде:

1. Написания отчета практики, дневника практики, аттестационного листа;
2. Размещения кода на GitHub для хранения и демонстрации;
3. Подготовки руководства пользователя.

Разработка форм для ввода, просмотра и редактирования данных

Таблица 10 — Главное окно приложения

Элемент	Тип	Назначение
Список задач	ListBox	Отображение всех задач в виде списка
Кнопка «Добавить»	Button	Открытие формы для создания задачи
Кнопка «Редактировать»	Button	Открытие формы для изменения задачи
Кнопка «Удалить»	Button	Удаление задачи
Кнопка «Обновить»	Button	Обновление списка задач
Кнопка «Сохранить»	Button	Сохранение задач в JSON-файл
Кнопка «Загрузить»	Button	Загрузка задач из JSON-файла
Кнопка «Статистика»	Button	Отображение статистики
Фильтр по статусу	ComboBox	Выбор статуса для фильтрации
Поле поиска	TextBox	Ввод ключевого слова для поиска

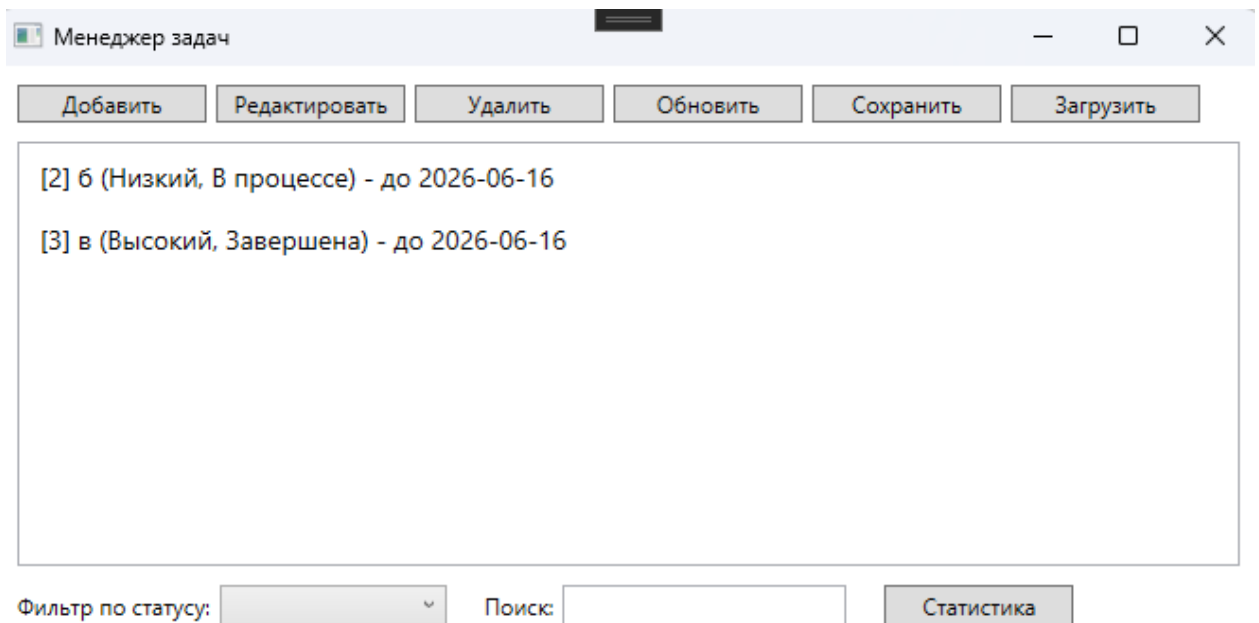


Рисунок 8 - Главное окно

Окно редактирования используется как для создания новой задачи, так и для изменения существующей.

Таблица 11 — Элементы окна редактирования

Элемент	Тип	Назначение
Название	TextBox	Ввод названия задачи
Описание	TextBox (многострочный)	Ввод описания задачи
Приоритет	ComboBox	Выбор приоритета
Срок	DatePicker	Выбор даты выполнения
Статус	ComboBox	Выбор статуса
Важная задача	CheckBox	Отметка о важности
ОК	Button	Подтверждение и сохранение
Отмена	Button	Отмена без сохранения

Редактор задачи

Название: a

Описание: a

Приоритет: Низкий

Срок: 16.06.2026 15

Статус: В процессе

☒ Важная задача ★

OK Отмена

Рисунок 9 - Форма редактирования

Этапы разработки интерфейса

Проектирование

На этапе проектирования была определена структура навигации:

Главное окно (MainWindow)

Добавить задачу — TaskEditWindow (новая)

Редактировать — TaskEditWindow (существующая)

Удалить — Диалог подтверждения

Сохранить — Сохранение в JSON

Загрузить — Загрузка из JSON

Фильтр — Обновление списка

Поиск — Обновление списка

Статистика — MessageBox с данными

Прототипирование

Был создан низкодетализированный прототип (эскиз) интерфейса для проверки:

1. Расположения элементов управления;
2. Логика переходов между окнами;
3. Визуальной иерархии и группировки элементов.
4. На основе прототипа были приняты решения:
5. Кнопки действий расположены в верхней части для быстрого доступа;
6. Список задач занимает основную область для наглядности;
7. Фильтр и поиск объединены в нижней панели для удобства.

Стилизация

Таблица 12 — Стилизация

Элемент	Оформление
Окно	Белый фон, стандартные элементы Windows
Список задач	Черный текст на белом фоне, разделители между задачами
Важные задачи	Символ звезды ★
Кнопки	Стандартные кнопки
Шрифты	Размер 14pt для читаемости
Отступы	Отступы 10px для аккуратного вида

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ

3.1. Процесс отладки

Отладка (debugging) — это процесс обнаружения, локализации и устранения ошибок (дефектов) в программном обеспечении. В ходе отладки разработчик последовательно выполняет следующие действия:

1. Воспроизведение ошибки — обнаружение некорректного поведения программы.
2. Локализация — определение места в коде, где возникает ошибка.
3. Анализ причин — выявление причин возникновения ошибки.
4. Устранение — исправление ошибки.
5. Проверка — подтверждение того, что ошибка устранена и не появились новые.

При разработке приложения «TaskManagerApp» использовались следующие инструменты отладки:

Таблица 13 — Инструменты отладки

Инструмент	Назначение
Visual Studio Debugger	Основной инструмент для пошаговой отладки
Breakpoints	Остановка выполнения программы в определенных местах для анализа состояния
Пошаговое выполнение	Последовательное выполнение кода с контролем каждой строчки
Окна просмотра	Просмотр текущих значений переменных и свойств объектов
Окна вывода	Просмотр отладочных сообщений и ошибок времени выполнения

Обработка исключений Отлов и анализ ошибок времени выполнения
Тестирование (xUnit) Автоматическая проверка корректности работы
методов

Процесс отладки приложения «TaskManagerApp»

Отладка разработанного приложения проводилась на всех этапах разработки по следующей схеме:

1. Написание кода.
2. Компиляция проекта → ошибки компиляции (исправляются сразу).
3. Запуск приложения → ошибки времени выполнения.
4. Воспроизведение ошибки (входные данные + шаги).
5. Установка точек останова в предполагаемом месте ошибки.
6. Повторный запуск в режиме отладки (Debug).
7. Пошаговое выполнение кода (F10/F11) и анализ значений переменных.
8. Выявление причины ошибки.
9. Исправление ошибки в коде.
10. Повторная проверка (перезапуск и тестирование).
11. Запуск модульных тестов для проверки корректности.

Пример ошибки, выявленной в процессе отладки

Ошибка CS0104 — неоднозначная ссылка на TaskStatus

Описание: При компиляции возникала ошибка:

CS0104: "TaskStatus" является неоднозначной ссылкой между "TaskManagerLib.Models.TaskStatus" и "System.Threading.Tasks.TaskStatus".

Процесс отладки:

1. Прочитано сообщение об ошибке в окне вывода.

2. Установлено, что имя TaskStatus конфликтует со встроенным типом из пространства имён System.Threading.Tasks.
3. Принято решение переименовать собственный enum в MyTaskStatus.
4. Выполнен поиск по всем файлам (Ctrl + Shift + F) слова TaskStatus.
5. Обнаружены оставшиеся упоминания в файлах TaskManager.cs, MainWindow.xaml.cs, TaskEditWindow.xaml.cs.
6. Все ссылки заменены на MyTaskStatus.

Исправление:

// Было:

```
public enum TaskStatus { ... }
```

// Стало:

```
public enum MyTaskStatus { ... }
```

Использование Breakpoints

В процессе отладки активно использовались точки останова для анализа работы программы.

Пример установки точки останова:

1. Открыт файл TaskManager.cs.
2. Кликнут левой кнопкой мыши на поле слева от номера строки (серая область) напротив метода AddTask.
3. Появился красный кружок — точка останова установлена.

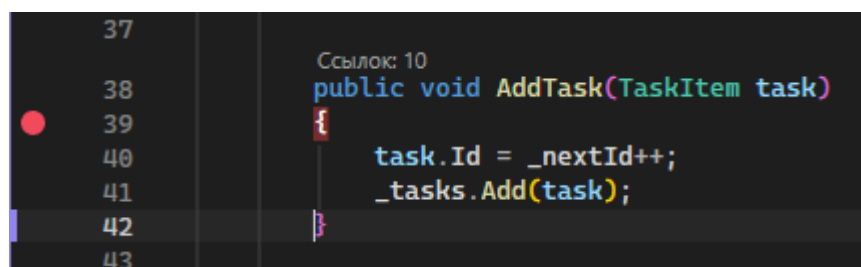


Рисунок 10 - Пример точки останова

Отладка с использованием модульных тестов

Помимо ручной отладки, использовались модульные тесты (xUnit) для проверки корректности работы отдельных методов.

Процесс отладки с тестами:

1. Написан тест для проверки конкретной функции.
2. Запущен тест через обозреватель тестов.
3. Если тест упал (красный цвет) — проанализирована ошибка.
4. В тесте установлена точка останова для пошагового выполнения.
5. Выполнен тест в режиме отладки (Debug Test).
6. Проанализированы значения переменных в процессе выполнения.
7. Исправлен код, тест запущен повторно.

```
[Fact]
Ссылка 0
public void AddTask_ShouldIncreaseCount()
{
    var manager = new TaskManager();
    var task = new TaskItem(0, "Тест", "Описание", TaskPriority.Средний, DateTime.Today, MyTaskStatus.Новая);

    manager.AddTask(task);
    var all = manager.GetAllTasks();

    Assert.Single(all);
    Assert.Equal(1, all[0].Id);
}
```

Рисунок 11 - Пример теста добавления задачи

Пример теста:

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования

Таблица 14 — Виды тестирования, применяемые в проекте

Вид тестирования	Описание	Применение в проекте
Модульное тестирование	Проверка отдельных компонентов (методов, классов) в изоляции	Тестирование всех классов TaskManager через xUnit
Интеграционное тестирование	Проверка взаимодействия между модулями	Тестирование связи между библиотекой и WPF-приложением
Функциональное тестирование	Проверка соответствия функциональным требованиям	Ручная проверка всех функций через интерфейс
Тестирование UI/UX	Проверка интерфейса на удобство и корректность отображения	Проверка отображения задач, работы фильтров, окон
Тестирование сохранения/загрузки	Проверка работы файлами	Проверка сериализации/десериализации JSON
Регрессионное тестирование	Проверка, что изменения не сломали функционал	Повторный запуск всех тестов после внесения изменений

Таблица 15 — Инструменты тестирования

Инструмент	Назначение	Версия
------------	------------	--------

xUnit	Фреймворк для написания модульных тестов	для xUnit.net 2.x
Visual Studio Test Explorer	Запуск и просмотр результатов тестов	Visual Studio 2026
Assert	Класс для проверки условий в тестах	Unit.Assert
Ручное тестирование	Проверка работы через-интерфейс	

Создание Unit-тестов

В решении TaskManagerApp был создан отдельный проект для тестов — TaskManagerTests. В этом проекте установлена ссылка на тестируемую библиотеку TaskManagerLib.

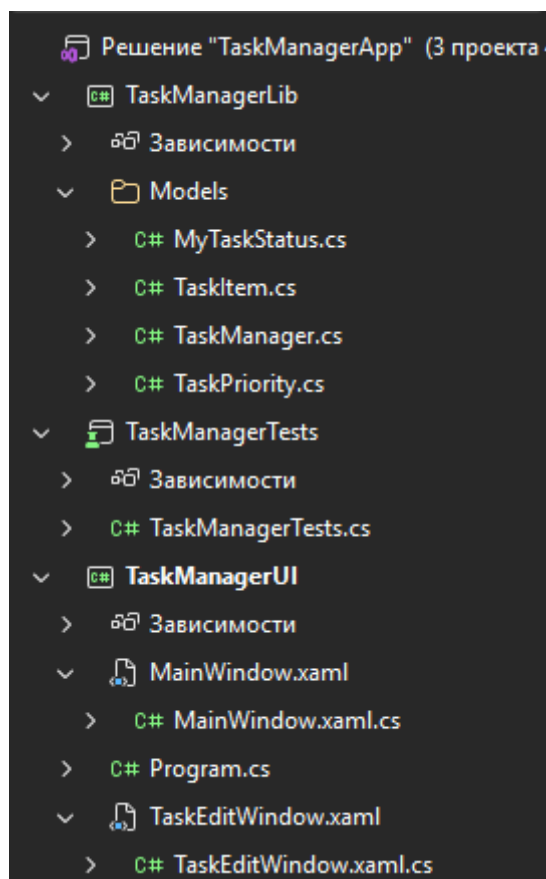


Рисунок 12 - Структура проекта

Настройка проекта:

1. Создан проект xUnit Test Project.
2. Добавлена ссылка на TaskManagerLib:
3. Правой кнопкой → Зависимости → Добавить ссылку.
4. Добавлен пакет xUnit через NuGet (устанавливается автоматически при создании проекта).

Пример разработанного теста

Проверка добавления задачи

[Fact]

```
public void AddTask_ShouldIncreaseCount()
{
    // Arrange (Подготовка)
    var manager = new TaskManager();
    var task = new TaskItem(0, "Тест", "Описание",
                           TaskPriority.Средний, DateTime.Today,
                           MyTaskStatus.Новая);

    // Act (Действие)
    manager.AddTask(task);
    var all = manager.GetAllTasks();

    // Assert (Проверка)
    Assert.Single(all); // В списке должна быть ровно одна задача
    Assert.Equal(1, all[0].Id); // ID должен быть присвоен автоматически
}
(1)
```

Что проверяется:

1. Задача добавляется в коллекцию;
2. ID присваивается автоматически (начиная с 1);
3. Количество задач увеличивается на 1.

Процесс запуска и анализа тестов

Запуск тестов в Visual Studio

Тесты запускаются через Обозреватель тестов (Test Explorer):

1. В меню: Тест → Обозреватель тестов (или Ctrl + E, T).
2. Открывается окно со списком всех тестов.
3. Для запуска всех тестов: нажать кнопку «Запустить все».
4. Для запуска конкретного теста: правой кнопкой по тесту → Запустить.

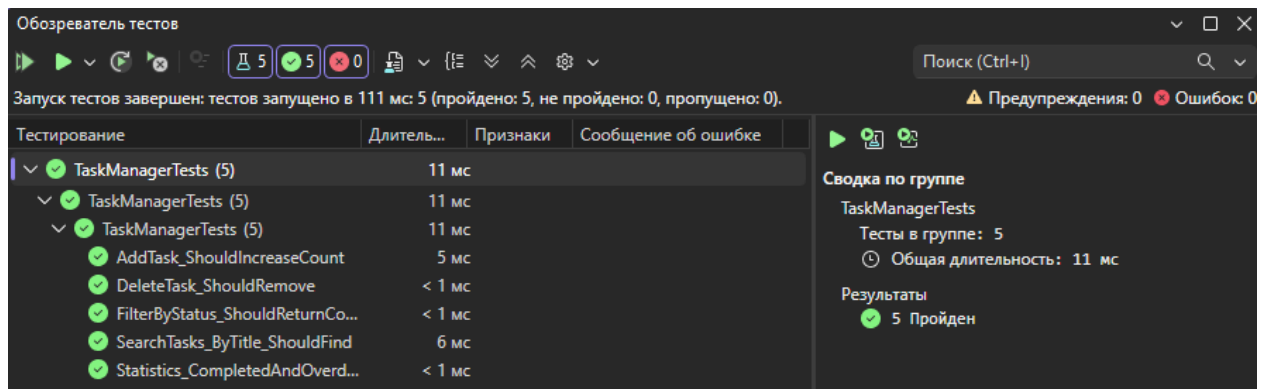


Рисунок 13 - Запуск тестов

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Определение инспекции кода

Инспекция кода — это систематическая проверка исходного кода программы с целью обнаружения и исправления ошибок, которые остались незамеченными в начальной фазе разработки.

В ходе разработки применялись следующие виды инспекции:

Таблица 16 — Виды инспекции

Вид	Описание	Применение в проекте
Самопроверка	Самостоятельная проверка	Проверка каждого написанного метода
Автоматизированная проверка	Использование инструментов анализа кода	Проверка через дляанализаторы Visual Studio
Инспекция на соответствие стандартам	Проверка соблюдения правил оформления	Проверка именования, форматирования, стиля
Технический анализ	Проверка архитектурных решений	Проверка на соответствие принципам ООП
Функциональный анализ	Проверка корректности работы логики	Сочетается с тестированием

Актуальные стандарты кодирования

При разработке соблюдались следующие стандарты:

Таблица 17 — Общие стандарты именования

Элемент	Правило	Пример
Классы	Имена существительные, PascalCase	TaskManager, TaskItem
Интерфейсы	Начинаются с «I», - PascalCase	
Методы	Глаголы, PascalCase	AddTask(), GetAllTasks()
Свойства	Имена существительные, PascalCase	TaskList, Priority
Поля	_camelCase, подчеркиванием	c_tasks, _nextId
Локальные переменные	camelCase	Task, filtered
Параметры методов	camelCase	NewTitle, newPriority
Перечисления (enum)	PascalCase	MyTaskStatus
Значения enum	PascalCase	Новая, ВПроцессе, Завершена

Пример использования:

```
public class TaskManager
{
    // приватное поле
    private List<TaskItem> _tasks = new List<TaskItem>();
    private int _nextId = 1;

    // метод + параметр
    public void AddTask(TaskItem task)
    {
        task.Id = _nextId++;
    }
}
```

```

        _tasks.Add(task);
    }

    // метод + тип возврата
    public List<TaskItem> GetAllTasks()
    {
        return _tasks.ToList();
    }
}

```

Таблица 18 — Стандарты форматирования кода

Правило	Описание	Пример
Отступы	Использование 4 пробелов	Настройка по умолчанию
Фигурные скобки	Открывающая скобка на новой строке	Public void Method()\n{
Интервалы	Пробелы между операторами	Int result = a + b;
Длина строки	Не более 120 символов	Строки переносятся при необходимости
Пустые строки	Между методами — 1 пустая строка	Для улучшения читаемости

Пример:

```

public bool UpdateTask(int id, string newTitle, string newDescription,
                        TaskPriority newPriority, DateTime newDueDate,
                        MyTaskStatus newStatus, bool newIsImportant)
{
    var task = _tasks.FirstOrDefault(t => t.Id == id);
    if (task == null) return false;

    task.Title = newTitle;
}

```

```

task.Description = newDescription;
task.Priority = newPriority;
task.DueDate = newDueDate;
task.Status = newStatus;
task.IsImportant = newIsImportant;
return true;
}

```

Таблица 19 — Программы, при помощи которых осуществляется инспекция

Инструмент	Назначение
Встроенные анализатор кода Visual Studio	Автоматическое обнаружение проблем
Окно «Список ошибок»	Отображение всех ошибок и предупреждений
Рефакторинг	Улучшение структуры кода
Git	Отслеживание изменений

Пример инспекции:

До инспекции:

```

public class taskmanager // Класс должен быть с большой буквы
{
    private List<TaskItem> tasks; // Приватное поле должно начинаться с
—
    public void addtask(TaskItem t) // Метод должен быть с большой
буквы
    {
        tasks.Add(t);
    }
}

```

```
}  
}
```

После инспекции:

```
public class TaskManager    // PascalCase  
{  
    private List<TaskItem> _tasks; // _camelCase  
  
    public void AddTask(TaskItem task) // PascalCase + осмысленное имя  
    {  
        _tasks.Add(task);  
    }  
}
```


ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Разработано приложение «TaskManagerApp» — программа для управления задачами с графическим интерфейсом, полностью соответствующая требованиям технического задания.
2. Реализован полный CRUD-функционал — добавление, просмотр, редактирование и удаление задач с возможностью фильтрации по статусу и поиска по названию/описанию.
3. Спроектирована архитектура приложения — бизнес-логика вынесена в динамическую библиотеку классов (TaskManagerLib), а пользовательский интерфейс реализован на WPF, что обеспечивает разделение ответственности и упрощает тестирование.
4. Написаны модульные тесты — с использованием фреймворка xUnit проверена корректность работы основных методов библиотеки (добавление, фильтрация, поиск, удаление, статистика); все тесты успешно пройдены.
5. Организовано хранение данных — реализована сериализация и десериализация задач в JSON-формат, что позволяет сохранять данные между сеансами работы.
6. Реализован дополнительный функционал — возможность пометки задачи как «Важная» (★), подсчёт статистики (количество завершённых и просроченных задач).
7. Проведена отладка и инспекция кода — выявлены и устранены все ошибки, код приведён в соответствие со стандартами оформления C#.

8. Подготовлена документация — составлена сопроводительная записка, включающая титульный лист, оглавление, диаграмму классов и подробное описание процесса разработки с иллюстрациями.

9. Исходный код размещён на GitHub — ссылка на репозиторий предоставлена руководителю практики.

По окончании практики был получен практический опыт разработки требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. Единая система программной документации (ЕСПД). Техническое задание на разработку программы или программного изделия. – М.: Стандартинформ, 1978.
2. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Стандартинформ, 1989.
3. ГОСТ 19.101-77. Единая система программной документации (ЕСПД). Виды программных документов. – М.: Стандартинформ, 1977.
4. Microsoft Corporation. Руководство по соглашениям об именовании в C#. – URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/fundamentals/coding-style/identifier-names> (дата обращения: 15.06.2026).
5. Microsoft Corporation. Создание WPF-приложения на C#. Документация по Visual Studio. – URL: <https://learn.microsoft.com/ru-ru/visualstudio/get-started/csharp/tutorial-wpf> (дата обращения: 15.06.2026).
6. Microsoft Corporation. Модульное тестирование C# в .NET с использованием dotnet test и xUnit. – URL: <https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-csharp-with-xunit> (дата обращения: 15.06.2026).
7. Microsoft Corporation. Отладка кода на C# с использованием Visual Studio Debugger. – URL: <https://learn.microsoft.com/ru-ru/visualstudio/get-started/csharp/tutorial-debugger> (дата обращения: 15.06.2026).

Требование к написанию программного модуля

Задание.

Цель:

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования: Разработать приложение для управления задачами с возможностью:

1.1 Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).

1.2 Просмотра списка всех задач.

1.3 Фильтрации задач по статусу (например, "Новая", "В процессе", "Завершена").

1.4 Поиска задач по названию или описанию.

1.5 Редактирования существующих задач.

1.6 Удаления задач.

1.7 Сохранения и загрузки задач в/из файла (JSON или XML).

1.8 Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.

1.9 Для классов в библиотеке необходимо написать модульные тесты.

2. Дополнительные задания (по желанию)

2.1 Реализовать сортировку задач по приоритету или сроку выполнения.

2.2 Добавить возможность пометки задачи как "Важная" (выводить звёздочкой или цветом в консоли).

2.3 Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Сопроводительная записка должна содержать:

- титульный лист,
- оглавление,
- диаграмму классов,